RESTful Web Service for Library Book Management

We have learned the fundamentals of REST in our course. In this assignment, you are going to implement a RESTful web service in Java language for the purpose of library book management. For simplicity, let's assume that the intended users are only the librarians. The web service should provide basic functionalities such as book lookup, loaning and returning. You are not required to implement a client application with a GUI.

FAQ of the project

WARNING: The project must be developed independently. No plagiarism is allowed. We use tools to detect clones of any materials.

Announcement

Good news! You can get a bonus of 0.5 (up to 3) if you can find any issue in this document.

Update on 10-MAR-2020: Project deadline for both Phase 1 and Phase 2 has changed. Please view below for the detailed schedule. The FAQ page is also being updated regarding your questions.

- Announcement
- Requirements and Test Cases
  - Authentication (Log In/Log Out)
  - Adding books
  - Book lookup
  - Book loaning
  - Book returning
  - Book deletion
  - Transactional behaviors
  - Concurrent operations
- Test Cases
- Phases, Milestones and Grading Policy
  - Phase 1 (Feb 19th – Apr 17th, 40%)
    - Milestone 1.1: Building the development environment
    - Milestone 1.2: Coding for the 6 function points (3% for each function point, 18% in total)
    - Milestone 1.3: Coding for the last 2 function points (6% for each function point, 12% in total)
    - Milestone 1.4: Bug bounty program (10%)
  - Phase 2 (**Apr 17th** – June 2nd, 60%)
    - Milestone 2.1: Building the development environment
    - Milestone 2.2: Refine the existing web service to a high-performance web service.
    - Milestone 2.3: Bug bounty program (10%)

Requirements and Test Cases

In the following, we list the requirements and provide some basic test cases. Please note that you should try your best to make the web service robust and we will evaluate it with many more test cases.

Authentication (Log In/Log Out)

A librarian needs to login using her username and password before she does some operations. In the system, we have **100 default users for Phase 1**, whose usernames are user001, user002, ..., user100, and passwords are pass001, pass002, ..., pass100.

A librarian sends the following request to the server to log in.

`POST` `http://localhost:8080/BookManagementService/login`

```
{
  "Username": "user001",
  "Password": "pass001"
}
```

If the username and password are correct, then an access token is returned. An access token1 is a magic string that will be used for authentication. Every time a librarian logs in, a unique access token is generated.

`HTTP/1.1 200 OK`

```
{
  "Token": "asf3219"
}
```

Note: The token can be randomly generated or computed by some encrypting functions. Make sure the token is unique and non-trivial, i.e., not easy to guess.

If one has logged in using the same username, the following will be returned:

`HTTP/1.1 409 Conflict`

If the username or password is not correct, the following will be returned:

`HTTP/1.1 400 Bad request`

To log out, the following request should be sent to the server:

`GET` `http://localhost:8080/BookManagementService/logout?token=asf3219`

If she successfully logs out, the access token then becomes deprecated and the following will be returned:

`HTTP/1.1 200 OK`

If she fails to log out, for example, the token is not valid, the following will be returned:

`HTTP/1.1 400 Bad request`

Adding books

You should use a database to store the following information on each book. For simplicity, let us assume there is only one copy of a book in the library.

- **ID:** a unique identifier of the book
- **Title:** the title of the book
- **Author:** the author of the book (let us assume there is only one author for each book)
- **Publisher:** the publisher of the book
- **Year:** the publication year of the book
- **Available:** indicating whether the book is available for loan

After login, the librarians should be able to add a new book by sending an HTTP request as below. The request body is in JSON format. You can add additional fields in a JSON object but you cannot delete any field we listed in the document. If you add additional fields in a JSON object, you cannot expect that our test cases contain your additional fields. That is, if we input a JSON object without your additional fields, your system should still work well and cannot throw exceptions or crash. The token in the URL is for authentication.

`POST` `http://localhost:8080/BookManagementService/books?token=asf3219`

```
{
  "Title": "Alice in Wonderland",
  "Author": "Lewis Carroll",
  "Publisher": "Macmillan Publishers",
  "Year": 1865
}
```

If the book does not exist in the database, it should be added and the following response should be returned:

`HTTP/1.1 201 Created`

`Location: /books/1`

Note: The second line Location: /books/1 should be in the HTTP response header, as its format indicates.

Otherwise, the following error message should be returned and point the user to the existing book record:

`HTTP/1.1 409 Conflict`

`Duplicate record: /books/1`

Note: The second line Duplicate record: /books/1 should be in the HTTP response header, as its format indicates.

Book lookup

The librarians should be able to look up books by providing any (or any combination) of the following information: ID, title, or author's name. For example, they can search for the added book by any of the following GET HTTP requests:

http://localhost:8080/BookManagementService/books?id=5&token=asf3219

http://localhost:8080/BookManagementService/books?title=Alice&token=asf3219

http://localhost:8080/BookManagementService/books?author=Lewis&token=asf3219

http://localhost:8080/BookManagementService/books?author=Lewis&id=5&token=asf3219

If there are no query parameters provided, the web service should return the list of all books in the library. If the searched book is found in the database, the web service should return the information of the book in JSON format like the following:

`HTTP/1.1 200 OK`

```
{
  "FoundBooks": 1,
  "Results": [{
    "Title": "Alice in Wonderland",
    "Author": "Lewis Carroll",
    "Publisher": "Macmillan Publishers",
    "Year": 1865
  }]
}
```

If there are multiple records found, the response body should contain the information of all found books and the total number of records. If there are no records found, the web service should return "204 No Content".

In addition to the above basic searching functionality, the web service should also support the following query parameters.

- `limit=N`: this parameter specifies the limit on the number of returned records. For example, setting the limit to 5 will make the web service return at most five records to the users after the query.
- `sortby=title&order=asc`: the `sortby` parameter specifies the criterion of ordering the returned records and its value can be id, title or author; the order parameter specifies whether the records should be sorted in an ascending (`asc`) or descending (`desc`) order.

For example, the following GET request tells the web service to return the information of the most recently added 10 books according to the order that they were added (let us assume the book id increases by 1 whenever a new book is successfully added):

http://localhost:8080/BookManagementService/books?limit=10&sortby=id&order=desc&token=asf3219

Book loaning

The librarians should be able to loan a book to readers by sending the following request to the resource URL of the book. For example, the following PUT request will tell the web service to update the status of the book with ID 1 to loaned:

PUT http://localhost:8080/BookManagementService/books/1?token=asf3219

```
{
  "Available": false
}
```

If the book does not exist in the database, the web service should return the following response:

`HTTP/1.1 404 No book record`

If the book is currently loaned to others (i.e., invalid status), the web service should return the following error message:

`HTTP/1.1 400 Bad request`

Otherwise, the web service should update the status of the book and return:

`HTTP/1.1 200 OK`

Book returning

Similar to book loaning, when readers return a book, the librarians should be able to reset the book's status to available by sending the following request to the resource URL of the book:

`PUT` `http://localhost:8080/BookManagementService/books/1?token=asf3219`

```
{
  "Available": true
}
```

If the book does not exist in the database, the web service should return the following response:

`HTTP/1.1 404 No book record`

If the book is currently not loaned (i.e., invalid status), the web service should return the following error message:

`HTTP/1.1 400 Bad request`

Otherwise, the web service should update the status of the book and return:

`HTTP/1.1 200 OK`

Book deletion

The librarians can delete a book record by sending DELETE request to the book's resource URL. For example, the following request asks the web service to delete the book with ID 1:

`DELETE` `http://localhost:8080/BookManagementService/books/1?token=asf3219`

If the book does not exist in the database, the web service should return the following response:

`HTTP/1.1 404 No book record`

Otherwise, the web service should delete the book record from the database and return:

`HTTP/1.1 200 OK`

Transactional behaviors

A transaction is a logical unit that is independently executed for data retrieval or updates. A transaction must be atomic, consistent, isolated and durable -- summarized as the ACID acronym. You can refer to the course slides for more details.

A librarian can perform a transactional behavior in three steps: request a transaction id, prepare a series of action plans, and commit or cancel the transaction. A transaction should be automatically canceled if it timeouts (e.g., there are no operations in 2 min).

Step 1: Request a Transaction ID

`OST http://localhost:8080/BookManagementService/transaction?token=asf3219`

If there is an available transaction id, the web service should return the id:

`HTTP/1.1 200 OK`

```
{
  "Transaction": 1234
}
```

Otherwise, the web service should return the following:

HTTP/1.1 400 Bad request

Step 2: Prepare Operations

In a transaction, we need to support book loaning and book returning. For example, a librarian prepares to accept the returning of Book 1 and loan Book 2 and Book 3 to the same reader. Assume that only if Book 1 is returned successfully and both Book 2 and Book 3 are available, then Books 2 and 3 can be loaned. Otherwise, no book will be returned and no book will be loaned. In such a transaction, we will send the following action plans to the web service.

`PUT` `http://localhost:8080/BookManagementService/transaction?token=asf3219`

```
{
  "Transaction": 1234,
  "Book": 1,
```

```
  "Action": "return"
}
```

`PUT` http://localhost:8080/BookManagementService/transaction?token=asf3219

```
{
  "Transaction": 1234,
  "Book": 2,
  "Action": "loan"
}
```

`PUT` http://localhost:8080/BookManagementService/transaction?token=asf3219

```
{
  "Transaction": 1234,
  "Book": 3,
  "action": "loan"
}
```

For each of the above three action plans, if the web service successfully records them, the following message is returned

`HTTP/1.1 200 OK`

Otherwise, the following message is returned.

`HTTP/1.1 400 Bad request`

Step 3: Commit or Cancel a Transaction:

`POST` http://localhost:8080/BookManagementService/transaction?token=asf3219

```
{
  "Transaction": 1234,
  "Operation": "commit" or "cancel"
}
```

If the transaction is committed or canceled successfully, the web service will return

`HTTP/1.1 200 OK`

Otherwise, it will return

`HTTP/1.1 400 Bad request`

Concurrent operations

We need to allow multiple librarians to perform the above operations at the same time.

Test Cases

Click here for detailed test rules and test cases for Phase 1.

Phases, Milestones and Grading Policy

To finish this assignment, you need to form a team with **2** people. If your decide to finish the project alone, please send an email to Tas and instructor for approval.

This assignment is split into two phases, each of which contains several milestones. It is not necessary for you to strictly follow the milestones, but you have to finish all tasks after each phase. We will check and grade your assignment after each phase.

Phase 1 (Feb 19th – Apr 17th, 40%)

In this phase, our goal is to design and develop a book management system satisfying the aforementioned requirements, based on an existing HTTP framework, **Apache HTTP Core 4.4** [2]. You **must** use **MySQL 5.7** [3] as the database to store and query data. **MySQL Connector/J 5.1** [4] **must** be the driver for database connection. Alternatively, you can use the JSON library **Jackson 2.10{**5] to process JSON data. Because we do not require to develop a client for the book management system, you **must** use **JMeter 5.x** [6] to test and demo your web service. You **must** use **GitHub** [7] and **Git**[8] to manage your project.

Milestone 1.1: Building the development environment

Before Feb 26th, you are expected to register your team and create a **private** code repository at GitHub. Each team can have at most **two** team members. Make sure you invite the TA account **comp4111ta** as a team member. You need to send an email on your team information, e.g., people and a URL to your code repository. You also need to get familiar with the development environment, e.g., how to implement a REST framework.

Milestone 1.2: Coding for the 6 function points (3% for each function point, 18% in total)

Before Mar 5th, you are expected to finish the first six function points, i.e., authentication, adding books, looking up books, loaning books, returning books, and deleting books.

Grading Policy for Each Function Point:

1. Correctly return requested items (e.g., book records), the status code (e.g., HTTP/1.1 404) and the status message (e.g., "No book records").—— 1%
2. The system cannot crash under any testing request.—— 1%
3. The code should be readable and follow a consistent coding style. —— 1%

Milestone 1.3: Coding for the last 2 function points (6% for each function point, 12% in total)

Before **April 6th 12:00AM**, you are expected to finish the last two function points, i.e., allowing transactional and concurrent operations. **The commits after that time will not**

count.

Grading Policy for Each Function Point:

1. Correctly return requested items (e.g., book records), the status code (e.g., HTTP/1.1 404) and the status message (e.g., "Bad request").—— 3%
2. The system cannot crash under any testing request.—— 2%
3. The code should be readable and follow a consistent coding style. —— 1%

Milestone 1.4: Bug bounty program (10%)

Organization: From **April 6th** to **April 17th**, we will hold code review competition, and the competition results will determine how many points you can get in the 10% points.

In this bounty program, the TA will randomly group two teams as a competition group. A team in a competition group should be able to access the GitHub repo of its opponent's project so that the code can be downloaded, built, and tested.

Rule: Suppose Team A and Team B are in the same competition group. The goal of Team A is to review the code of Team B and find as many bugs as possible. Whenever Team A finds a bug in the code of Team B, they need to report an issue to Team B via GitHub's issue tracking system. The developers of Team B are responsible for **labelling** the issue as **bug**, **needmoreinfo**, or **invalid** status. Team B then needs to respond to the issue: fix the issue via pull request if it is a real issue, or directly close the issue report if it is not a real issue.

In this bounty program, we encourage you to use a static program analysis tool, called **Infer**9, to help you automatically scan the code and find bugs. We will provide a separate document to describe how to use this tool. When you report an issue found by Infer to your opponent, you need to emphasize that "Oh, this is a bug found by Infer" and provide a screenshot of Infer as the evidence. You will then get a bonus in your grade.

We take the following five kinds of bugs into consideration:

Aa Bug Kind	# Weight	Rule
Using wrong libraries	1	For example, he uses MySQL 5.8 or MongoDB instead of MySQL 5.7.
Exceptions found by manual code review	1	The issue reporter must be able to provide a test case to trigger the exception.
Data inconsistency	1	The issue reporter must be able to provide a test case to trigger the issue. For example, it triggers an inconsistency if a book can be loaned twice without returning.

Bug Kind	Weight	Rule
Exceptions found by Infer	1.5	The issue reporter must be able to provide a test case to trigger the exception.
Security Issues	3	A librarian can add/delete/loan/return books without login.A librarian can add/delete/loan/return books using an expired token (Once logout, the token is expired).A librarian can enforce other librarians to log out.For other security-like issues, please ask the TA to confirm.

Grading Policy: Assume Team A finds ***Ni*** (***i***=1,2,3,4,5 standing for the five kinds of bugs listed in the above table) real issues in the code of Team B. Assume Team B finds ***Mi*** real issues in the code of Team A. Assume the weight of each issue type is ***wi***. Then, Team A will get ***∑Ni*wi/(∑Ni*wi + ∑Mi*wi)*10%*** points, and Team B will get the remaining points.

In case that both teams cannot find any bug in their opponent's code, the TAs will try to find bugs in their code. If TA can find more than 3 real bugs in Team A's code, Team B will get 0 points in this competition. If TA cannot find more than 3 real bugs in both teams, each team will get 5 points in this competition.

Phase 2 (**Apr 17th** – June 2nd, 60%)

In this phase, our goal is to improve the performance of your existing API service implementation based on **Apache HTTP Core 4.4**. You can try various I/O strategies in this library to handle HTTP requests. The final goal is to develop a service that is as efficient as possible. We will have a performance competition at the end of this phase.

Milestone 2.1: Building the development environment

Before **Apr 20th**, we need to prepare and get familiar with Apache HTTP Core's advance features and various I/O strategies. You also need to create **10,000 default users for Phase 2**, whose usernames are user00001, user00002, ..., user10000, and passwords are pass00001, pass00002, ..., pass10000.

Milestone 2.2: Refine the existing web service to a high-performance web service.

Before **May 25th**, we continue to code our own RESTful framework based on HTTP Core 4.4. Together with the code, we write before, the whole system needs to satisfy the requirement of the book management system.

Grading Policy for Each Requirement:

1. The web service must be implemented to be **multi-threaded** for better performance. (15%)
2. The web service can handle **asynchronized** HTTP requests at the same time. (15%)

3. All 8 function points in Milestone 1 must work properly. (10%)

Milestone 2.3: Bug bounty program (10%)

Organization and Rules: From **May 25th to June 2nd**, we will hold a many-to-many bug bounty program, in which each team can review the code of all other teams. The bugs we take into consideration are the same as those in Milestone 1.4. In case multiple teams found the same bug in one team, we only count the bug for the first ream that reports the bug.

Grading Policy: After the program, we will compute a score for each team as follows and rank all teams according to the score.

Suppose Team A finds ***Ni*** (***i***=1,2,3,4,5 standing for the five kinds of bugs listed in the above table) real issues for other teams and other teams find ***Mi*** real issues in the code of Team A. Then, the score of Team A will be $-\sum Ni*wi+\sum Mi*wi$.

The teams will be ranked based on their scores in ascending order. A team will be graded according to its rank as below. We will rearrange the table according to the number of teams in this course.

Aa Rank	Grade
1	10%
2	9%
3	8%
...	
The last one	0

Milestone 2.4: Performance competition (10%)

Organization: From **May 25th to June 2nd**, we will hold a performance competition. Here, owing to Milestone 2.3, we assume that your opponent's system works well for the requirements and does not have any functionality issues.

Rule: TA will give everyone a server account to run the test suite on a powerful workstation to test the project code as soon as Milestone 2 kicks off. Before the submission deadline, you can always refine to improve the performance.

The performance of your system will be recorded and ranked in ascending order when the competitions is over.

Test Suite:

- **Concurrency Test** (4%):
    - 10,000 different users log in at the same time. All can receive success responses with access tokens.

    - A new book is added by one user. 10,000 users loan the same book simultaneously. Only one user has the success response. Others should go HTTP/1.1 400 Bad request.
    - 100 books from the year 1900 to 2000 are added. 10,000 users lookup all books in both ascending or descending orders simultaneously. All can have correct book results.
- **Throughput Test** (3%):
    - 100 books from the year 1900 to 2000 are added. 10,000 users lookup all books in both ascending or descending orders simultaneously.

The average response time of your system will be recorded and ranked in ascending order.

- **Fairness Test** (3%):
    - 100 books from the year 1900 to 2000 are added. 10,000 users lookup all books in both ascending or descending orders simultaneously.

The standard deviation of response time of your system will be recorded and ranked in ascending order.

Grading Policy: A team will be graded according to the following rules:

- Team A passes *Ci* concurrency tests. (***Ci*∈{1,2,3}**)
- Team A's average response time is ***Ti***, the fast average time among all teams is ***T0***.
- Team A's standard deviation of response time is ***Si***, the lowest one among all teams is ***S0***.

Final Score for Team A: ***Ci/3×4%+Ti/To ×3%+Si/S0 ×3%***

The test suite will be released as soon as Milestone 2.2 kicks off.

---

1. https://en.wikipedia.org/wiki/Access_token ↩
2. HTTP Core 4.4: https://hc.apache.org/httpcomponents-core-4.4.x/tutorial/html/index.html You cannot use other libraries for network communication. ↩
3. MySQL 5.7: https://dev.mysql.com/doc/refman/5.7/en/installing.html ↩
4. MySQL Connector/J 5.1: https://dev.mysql.com/downloads/connector/j/5.1.html ↩
5. Jackson 2.10 https://github.com/FasterXML/jackson/wiki/Jackson-Release-2.10 ↩
6. JMeter 5.0: https://jmeter.apache.org/ ↩
7. GitHub: https://github.com/ ↩
8. Git: https://git-scm.com/ ↩
9. Infer: https://fbinfer.com/ ↩